Belief and Empathy Particles

Bramblett and Bezzo's Heterogeneous Team, Recreated
as an Epistemic Planning Task on ePlanSys

Haniel Ulises

Epistemic Robotics

October 2026

Bramblett and Bezzo [1] propose a framework in which the robots of a heterogeneous team, out of radio contact, keep belief particles for one another, a robot with a fault keeps to the particle the others expect of it, and a robot that finds a task seeks the teammate it needs at that teammate's particles. They write the belief updates in dynamic epistemic logic and allocate tasks with a genetic algorithm. This report recreates the scenario of their Figures 1 and 6, with an aerial vehicle and two ground vehicles in Gazebo, and states it as an epistemic planning task for ePlanSys: the particles are the worlds of a Kripke model, a hail at a particle is an event model, and the gossip is a knowledge precondition. Aletheia's policy, of depth 10 over the six fault patterns, seeks the nearer vehicle at its first and second particles and, when both are empty, knows without a visit that the vehicle has two faults and that the other has none. The paper's update rules hold as formulas along the policy in every world; read with oblivious observers, as the paper's text suggests, the same task has no policy with consistent beliefs. Over 24 seeds under five formulations, every formulation did the task in every run. The policy and the paper's procedure finished within 1 s of each other in 12 of 24 seeds, the procedure 9.3 s sooner on average, from the seeds in which one vehicle had a fault, where its makespan estimate sees a difference a planner counting steps cannot; the ideal method was 37.5 s sooner and the flock 72.3 s later. In a variant with latent tool faults, which the particles cannot represent, Aletheia's policy is 65 s slower than the strongest version of the procedure. A timed search over the same epistemic task, called through ePlanSys's planner plugin, is level with the paper's procedure (1.9 s sooner on average, the paired 95 % interval from -8.0 to 4.2 s) in the replica, and in the variant level with the strong procedure, which its value shows to be within 1.2 s of the least expected time any policy built from the same moves over the same knowledge reaches in the kinematic model.

Contents

1 Scope 2

2 The paper 2

3 The paper's constructs in ePlanSys 2

4 Environment 3

5 Domain 3

6 The policy 4

7 Knowledge, step by step 5

8 The oblivious reading 5

9 Five formulations 6

10 Measurement 6

11 Evaluation over 24 seeds 6

12 A variant: latent tool faults 7

13 The least expected time 10

14 Larger teams 12

15 Implementation defects 12

16 Limitations 13

1 Scope

This report accompanies the page *Belief and empathy particles* of the Epistemic Robotics project and the package `belief_empathy_demo`. It summarises the paper, sets out what each of its constructs is in ePlanSys, states the domain and the policy, checks the paper's update rules along the policy, shows what the oblivious reading of its rules does, and reports a batch of 120 runs in Gazebo, a timed search over the same epistemic task with the least expected time it computes as a bound on every formulation, and a planning-only study of larger teams.

2 The paper

Problem 4.1 of [1] asks for a distributed policy for N_r robots with N_k kinds of capability that searches a cluttered, partly unknown area with limited communication for N_t tasks, each needing a subset R_τ of the capabilities, minimising mission time and letting the team cooperate when a robot's abilities change out of contact. Each robot i keeps particles $\mathcal{P}_i = \{p_{ij,b}\}$: belief particles for every other robot and empathy particles $p_{ii,b}$, its estimate of what the others believe of it, ordered by likelihood; a robot with a fault tracks its next particle. In contact the robots split the frontier by a generalised Dirichlet tessellation weighted by capability, their equation (3); out of contact each computes it over the common belief set $\mathcal{C}_i$. Belief updates are action models of dynamic epistemic logic [3], their equations (8) to (13), and tasks are allocated by a nonlinear integer program, equations (14) to (16), solved by a genetic algorithm warm-started by their Algorithm 1, of complexity $O(nm^2)$. The framework builds on the authors' earlier prediction and implicit coordination [2].

Their simulations use two to eight aerial and ground vehicles in fifteen random 50×50 m areas, three particles per robot at 100, 60 and 20 % of the vehicle's speed, a 10 m radio range and zero, one or two faults. Their Table I gives the mean mission time of the proposed method as 40.2, 48.6 and 53.1 s for zero, one and two faults, against 28.0, 30.4 and 32.9 s for an ideal method with unlimited communication and 113.4, 156.8 and 179.3 s for a flock method that keeps every robot in range.

The recreation here keeps the scenario of their Figures 1 and 6: one aerial vehicle u, two ground vehicles g_1 and g_2, one task that needs u and a ground vehicle together, three particles at the same speeds, and zero, one or two faults in the team. It departs from the paper where Table 1 says.

the paper	here
frontier partition recomputed over the common belief set	a partition computed once at the start, in proportion to speed, and fixed coverage routes every robot knows
faults at any time	faults before the task can be found, since the policy is computed when it is
a robot with faults is slower	a ground vehicle with two faults can no longer do a task
allocation by a genetic algorithm	the makespan estimate evaluated for both assignments; and Aletheia's policy
mission time includes covering the area	mission time ends when the task is done
MATLAB; ROSbots and a Crazyflie	Gazebo 11 and ROS 2: two TurtleBot3 Burgers and a quadrotor flown in the plane

Table 1: Where the recreation departs from the paper.

3 The paper's constructs in ePlanSys

ePlanSys plans in the logic the paper writes its updates in: a state is a pointed Kripke model, an action an event model applied by the product update [4], and a policy is searched for over the

models it reaches [5]. Table 2 sets out the correspondence.

in the paper	in `belief-empathy.epddl` and ePlanSys
epistemic state (W, R_i, V, W_d), Def. 1	the initial model built by plank from a finitary S5 theory: 12 worlds, 6 designated, those u considers possible
belief particles $p_{ij,b}$	the atoms $p1(g), p2(g), p3(g)$; a world is a fault pattern
empathy: a robot with a fault tracks its next particle	the identity of "g stands at particle b" with "g has had $b-1$ faults", on which the hail's events rest; and $K_g \neg K w_u\, p1(g)$, true at the start
common belief set $\mathcal{C}_i$	routes, speeds and the fault budget, common knowledge
$announce(\Omega)$, eq. (9)	a public event, after which the announcement is common knowledge; eq. (9) states its second level
$perceive(\neg track(p_{ij,b}))$, eq. (11)	the hail's absent event, after which $K_u \neg pb(g)$
$B_i\, track(p_{ij,b+1})$, eq. (12)	no counterpart: S5 models have no plausibility order, so u holds both later particles possible and the policy branches on both
$perceive(present(\tau))$, eq. (13)	*task* true, known to u, not known whether by either ground vehicle
gossip	the hail's met event, whose precondition holds *task*; and the precondition K_g *task* of the drive to the site
disconnected robots' knowledge "not affected"	observers *unsure* of what they do not see; the oblivious reading leaves the vehicle hailed believing a contradiction (Section 8)
capabilities R_τ, $\mathbf{I}_i^k$	*uav*, *ugv* and $p1(g) \vee p2(g)$ in the preconditions
allocation by a genetic algorithm, Algorithm 1	AO* over the product update: whom to seek, where, and whom to send, in one policy of least worst-case depth, without durations
a policy as a sequence, recomputed at each event	a conditional policy computed once, branching on what each hail returns
undecidability of games with imperfect information	planning in DEL is undecidable in general [5]; this task is finite, and AO* explores it exhaustively

Table 2: The paper's constructs and their counterparts.

4 Environment

The area is 30×16 m (Figure 1), crossed by four streets and four avenues, with obstacles in the blocks between them. u is a quadrotor flown at 1.8 m over the obstacles at 0.5 m/s; the particles of g_1 and g_2, TurtleBot3 Burgers, move at 0.16, 0.096 and 0.032 m/s. In contact at the start the robots split the area in proportion to speed, the west strip to g_1, the east to g_2 and the middle, twice as wide, to u, and each covers its strip along a known route. The radio reaches 2.5 m. The task stands at one of four sites on u's streets and needs u and a ground vehicle with fewer than two faults within 1 m of it for 8 s. Faults happen between 20 and 50 s after the start; a vehicle with a fault goes to its next particle, back along its route if need be, and holds it. Before any run it is checked that every way a ground vehicle drives clears the obstacles by 1.00 m, that a vehicle's particles are at least 4.0 m apart whenever a hail can happen, more than the radio's range, and that every site is found after the last fault.

5 Domain

The agents are u, g_1 and g_2. The atoms are the types *uav* and *ugv*, the half each robot is in, *task*, the particles $pb(g)$, u's position relative to the particles, the robots at the site, and *done*. u reaches a vehicle's first particle directly in the vehicle's half, and the second and third only from the one before, where it has hailed and found nobody. A hail has two designated events:

```
(:event e-met-1 :parameters (?u ?g - agent)
  :precondition (and (uav ?u) (by1 ?g) (p1 ?g) (task))
  :effects (:and (free ?u) (not (by1 ?g))))
(:event e-absent-1 :parameters (?u ?g - agent)
```

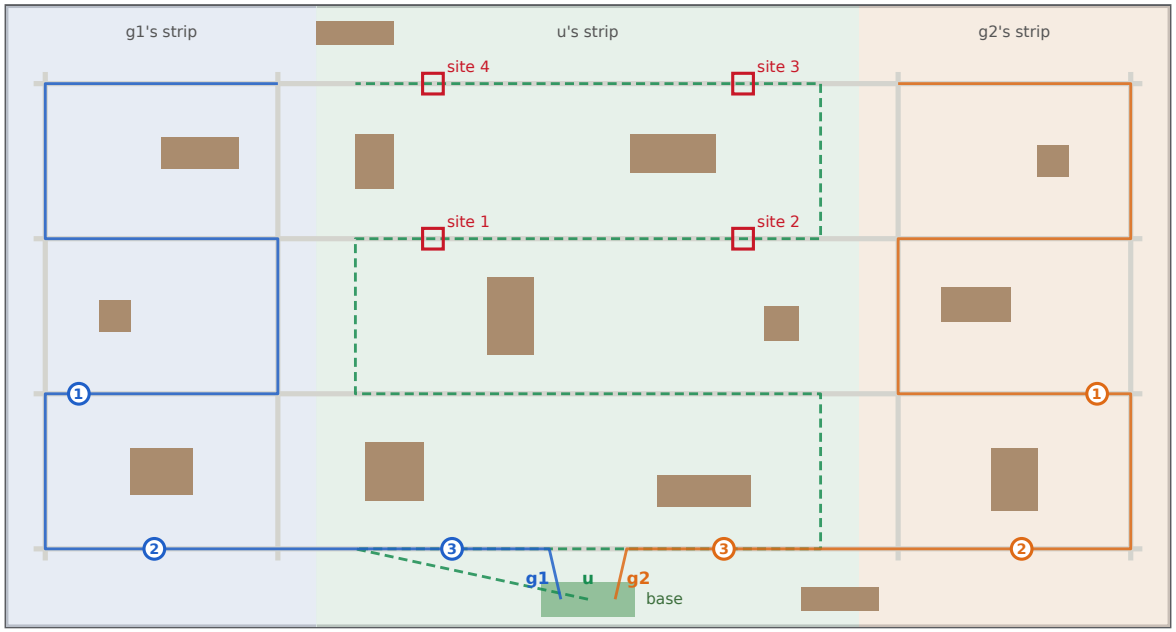


Figure 1: The area: each robot's strip and route, the four sites, and each ground vehicle's three particles 120 s after the start.

```
  :precondition (and (uav ?u) (by1 ?g) (not (p1 ?g)))
  :effects (:and (free ?u) (not (by1 ?g)) (passed1 ?g)))
(:action hail-p1 :parameters (?u ?g - agent | (/= ?u ?g))
  :action-type (hail (e-met-1 ?u ?g) (e-absent-1 ?u ?g) (nil))
  :observability-conditions
    (:and (?u Fully) (?g (if (p1 ?g) Fully else Unsure)) (default Unsure)))
```

The type `hail` has a null event, and `Unsure` relates every event to every event. The drive to the site, go-site(g), requires $K_g\,task$ and $p1(g) \vee p2(g)$; the task, perform(u, g), requires u and g at the site. The goal is *done*. The initial model has 12 worlds, the six fault patterns with at most two faults each with *task* true or false; u knows whether *task*, each vehicle knows only its own particle, and the six worlds with *task* are designated.

6 The policy

With the site in the east half, Aletheia's AO* [7] with consistent beliefs required returns, at depth 10 after 5 848 expansions:

```
fly-p1(u, g2)
hail-p1(u, g2)
  answered: fly-site(u)
  go-site(g2)
  perform(u, g2)
  no answer: back-p2(u, g2)
  hail-p2(u, g2)
    answered: fly-site(u)
    go-site(g2)
    perform(u, g2)
    no answer: cross-west(u)
    fly-p1(u, g1)
    hail-p1(u, g1)
    fly-site(u)
    go-site(g1)
```

```
perform(u, g1)
```

Proposition 1. After hail-p1(u, g_2) and hail-p2(u, g_2) both return their absent event, $K_u\, p3(g_2)$ and $K_u\, p1(g_1)$ hold.

Proof. Every world satisfies exactly one of $p1(g_2)$, $p2(g_2)$, $p3(g_2)$, by a common-knowledge formula of the initial theory that no event changes. u observes both hails fully, so every world it considers possible after them is the product of a world with the two absent events, whose preconditions are $\neg p1(g_2)$ and $\neg p2(g_2)$; each therefore satisfies $p3(g_2)$. The fault budget, also common knowledge, excludes $p3(g_2) \wedge \neg p1(g_1)$, so each also satisfies $p1(g_1)$. □

The policy therefore never flies to a third particle: by the proposition, the vehicle there cannot do the task, and the hail at g_1's first particle that follows has only its met event in the worlds u considers. With the site in the west the policy is the same with the vehicles swapped. The policy fixes what the paper's Algorithm 1 and genetic algorithm produce, an order of visits and an assignment, and with it the two conclusions of the proposition, which the paper's rules leave to the designer. It is a joint policy over common knowledge, which every robot could compute for itself, as in implicit coordination [6].

7 Knowledge, step by step

The policy was replayed through the product update in each of the six worlds and the paper's rules evaluated as formulas at the actual world after every step. In the world where g_2 has had two faults and g_1 none:

step	$K_{g2}p3(g2)$	$K_{g2}\neg Kw_u p1(g2)$	$K_u\neg p1(g2)$	$K_u p3(g2)$	$K_u p1(g1)$	$K_{g1}task$	$K_uK_{g1}task$	$\neg K_{g2}\neg K_{g1}task$	*done*
initial state	yes	yes	–	–	–	–	–	–	–
fly-p1(u, g2) / e-fly-p1	yes	yes	–	–	–	–	–	–	–
hail-p1(u, g2) / e-absent-1	yes	–	yes	–	–	–	–	–	–
back-p2(u, g2) / e-back-p2	yes	–	yes	–	–	–	–	–	–
hail-p2(u, g2) / e-absent-2	yes	–	yes	yes	yes	–	–	–	–
cross-west(u) / e-cross-west	yes	–	yes	yes	yes	–	–	–	–
fly-p1(u, g1) / e-fly-p1	yes	–	yes	yes	yes	–	–	–	–
hail-p1(u, g1) / e-met-1	yes	–	yes	yes	yes	yes	yes	yes	–
fly-site(u) / e-fly-site	yes	–	yes	yes	yes	yes	yes	yes	–
go-site(g1) / e-go-site	yes	–	yes	yes	yes	yes	yes	yes	–
perform(u, g1) / e-perform	yes	–	yes	yes	yes	yes	yes	yes	yes

Table 3: What each robot knows after each step, at the actual world, where g_2 has had two faults and g_1 none.

In all six worlds each vehicle knows its own particle throughout; at the start each knows $\neg Kw_u$ of its first particle, the empathy state; after an unanswered hail u knows the vehicle is not there, eq. (11), and holds both later particles possible; after a meeting $K_uK_g\, task$, while the other vehicle cannot rule out $K_g\, task$; and no agent considers no world possible. `tools/validate.sh` asserts these 50 checks.

8 The oblivious reading

The paper states that after a perception "the knowledge of disconnected robots is not affected". As an observability this is the oblivious type: from every event an agent considers only the null event possible.

Proposition 2. With g_2 oblivious of fly-p1(u, g_2) and fully observing the met event of hail-p1(u, g_2), g_2 considers no world possible after the hail.

Proof. After the flight, every world g_2 considers possible is a product with the null event, and $by1(g_2)$, which only the flight makes true, is false at each. At the hail g_2 observes the met event fully, so the worlds it considers possible after it are products of those worlds with the met event, whose precondition holds $by1(g_2)$. There are none. □

With consistent beliefs required, Aletheia finds no policy for the oblivious variant. Without that requirement it returns one of depth 8, which, replayed, leaves the vehicle met believing a contradiction in 5 hails, and in one world sends g_1 to the site where, at the actual world, it has not been told of the task. The domain makes a robot out of contact unsure of what it does not see: it learns nothing, and believes nothing false, so the true world stays accessible to every robot, the paper's eq. (8). The empathy rule is what makes this enough.

9 Five formulations

ePlanSys, Aletheia. Aletheia plans over the instance for the half the site is in when u finds the task, and ePlanSys's epistemic executor runs the policy, choosing each branch by the event the hail reports.

ePlanSys, timed. The timed search of Section 12 plans over the same instance, through the same planner plugin, told where the site is, and the same executor runs its policy.

The paper's procedure. u estimates, for each ground vehicle, the makespan of seeking it at its most likely remaining particle and of both then reaching the site, the vehicle at the speed of that particle, and seeks the least. After every hail it estimates again with what it learned, and assigns the task to the vehicle it met or sends it on. With one task and two vehicles the genetic algorithm's search space has two assignments, and both are evaluated. The procedure keeps a vehicle among the candidates until it learns that it cannot do the task.

Ideal. Every robot hears every other: the vehicle of least makespan is told at once.

Flock. The team sweeps the whole area in two bands, each ground vehicle 2 m from u, at the speed of its slowest member; a vehicle with two faults leaves the flock; on finding the task, the vehicle of least makespan goes.

Every formulation drives the robots through the same requests and controllers, and a ground vehicle drives to the site only on a site it has heard on the radio.

The paper recomputes the frontier partition as the robots cover; the recreation fixes it. On the time to the task this leaves the procedure no weaker: out of contact the partition is computed over the common belief set, which a fault does not change until the robots meet (eq. (10) updates it only on an announcement), so it cannot move the sound vehicle toward the site before that vehicle is hailed, and once hailed it is sent directly. The re-partitioning bears on coverage, which is not measured here.

10 Measurement

Mission time is the simulated time from the common start to the task done, judged by the simulator's poses. Every hail is recorded with its particle and outcome; a meeting at a particle the vehicle did not hold would be counted against the model, and there were none.

11 Evaluation over 24 seeds

The 24 seeds hold each fault pattern at each site once, and each ran under all five formulations from the same start (Tables 4 to 6).

Both Aletheia's policy and the procedure did the task in every run, and both hailed only 1.7 particles per run on average. They differed in 12 of 24 seeds, the policy sooner in 4 and later in 8. In 4 of those the procedure, after a hail found nobody, estimated again and went for the other vehicle, which it believed faster, where the policy followed the first vehicle's route back to its

formulation	done	mean (s)	worst (s)	search (s)	hails	third
ideal	24/24	219.3	304.3	0.0	–	–
ePlanSys, Aletheia	24/24	256.8	350.4	42.3	1.67	0
ePlanSys, timed	24/24	245.6	358.9	50.6	1.67	0
the paper's procedure	24/24	247.5	358.3	53.3	1.67	0
flock	24/24	329.1	430.6	0.0	–	–

Table 4: Per formulation, over all seeds. Search: from the task found to the vehicle that did it knowing of it. Third: runs with a hail at a third particle.

formulation	0	1	2	paper 0	paper 1	paper 2
ideal	199.2	215.0	228.8	28.0	30.4	32.9
ePlanSys, Aletheia	206.9	257.7	272.8	–	–	–
ePlanSys, timed	206.8	236.8	264.4	–	–	–
the paper's procedure	206.8	235.2	269.3	40.2	48.6	53.1
flock	247.2	384.5	319.4	113.4	156.8	179.3

Table 5: Mean mission time in seconds by the number of faults, beside the paper's Table I (its proposed method in the row of the procedure). The paper's times are for a different area, team and measure, and are given for their trend, not for comparison of values.

second particle, which is as deep in steps. Averaged over all seeds the procedure was 9.3 s sooner. Neither ever hailed a third particle: the policy because it knows a vehicle there cannot do the task, the procedure because a vehicle that slow is never worth it in its estimate. The two reasons coincide in this scenario, and the variant below separates them.

The ideal method, with nothing to search for, was 37.5 s sooner than Aletheia's policy on average: the price of being out of contact. The flock was 72.3 s slower than Aletheia's policy, and sooner in 3 seeds. With one fault it slows to the faulted vehicle's speed, 384.5 s on average; with two it is quicker again, 319.4 s, because a vehicle with two faults leaves it and the rest go on at full speed. The order of the paper's Table I holds, ideal before the method out of contact before the flock, but with a smaller margin to the flock: the area here takes two passes, and the time stops at the task where the paper's also covers the area.

The timed search, over the same task and through the same plugin, planned in 0.36 s against Aletheia's 1.2 s and took 245.6 s on average, 358.9 s at worst: level with the paper's procedure (1.9 s sooner on average, the paired 95 % interval from −8.0 to 4.2 s), sooner in 1 seed, within 1 s in 19 and later in 4. It weighs each continuation over the fault patterns still possible, where the procedure estimates one hail at a time as if the vehicle were at the particle hailed; the kinematic model put the difference at 3.1 s over every pattern at every site. Against Aletheia, the same knowledge without durations, it was 11.2 s sooner on average.

12 A variant: latent tool faults

In the paper and in the recreation, a fault slows a vehicle and the slowness is where the vehicle is. A vehicle on its third particle is slow and, here, unable, so the paper's makespan estimate avoids it for the reason the policy's inference does. The variant separates the two. A drive fault slows a vehicle, which tracks its next particle, and costs it nothing else. A tool fault leaves speed and particle as they were and is known to nobody, the vehicle included, until the vehicle attempts the task, when the attempt fails; a tool not exercised before the task cannot be checked before it. At most two faults of either kind happen, and at least one tool works: twelve patterns. The attempt $\mathsf{perform}(u, g)$ is an action with two events, observed by u and g:

$$e_{done} : tool(g) \to done, \qquad e_{fail} : \neg tool(g) \to tried(g),$$

and nothing else in the domain observes *tool*.

faults (g_1, g_2); half	ideal	Aletheia	timed	procedure	flock
0, 0; west	194	207	207	207	272
0, 0; east	205	207	207	207	222
1, 0; west	227	300	276	271	427
1, 0; east	205	207	207	207	340
0, 1; west	194	207	207	207	424
0, 1; east	234	317	257	257	347
2, 0; west	238	293	275	271	315
2, 0; east	204	207	207	207	258
0, 2; west	194	207	207	207	311
0, 2; east	249	314	257	257	257
1, 1; west	248	299	303	337	428
1, 1; east	240	316	337	337	347

Table 6: Mean mission time in seconds per fault pattern and half of the area, two seeds each.

On time, the strongest version of the procedure beats Aletheia's policy: 261.0 s against 326.2 s on average and 431.9 s against 581.2 s at worst. A timed search over the same epistemic task matches the procedure: 265.1 s on average and 423.6 s at worst, within 1 s of it in 11 of 24 seeds. The procedure as published, without the joint budget and the assumption that some tool works, took 273.3 s, 12.3 s more than the strong one: that difference is what the joint model contributes.

Aletheia's policy, of depth 12 after 36 136 expansions, recruits both vehicles: the first met is sent to the site, and u seeks the second. With the site in the east:

```
fly-p1(u, g2)
hail-p1(u, g2)
  answered: go-site(g2)
  cross-west(u)
  fly-p1(u, g1)
  hail-p1(u, g1)
    answered: fly-site(u)
    go-site(g1)
    perform(u, g1)
      no answer: perform(u, g2)
    no answer: back-p2(u, g1)
    hail-p2(u, g1)
      answered: fly-site(u)
      go-site(g1)
      perform(u, g1)
        no answer: perform(u, g2)
      no answer: fly-site(u)
      perform(u, g2)
  no answer: back-p2(u, g2)
  hail-p2(u, g2)
    answered: go-site(g2)
    cross-west(u)
    fly-p1(u, g1)
    hail-p1(u, g1)
      answered: fly-site(u)
```

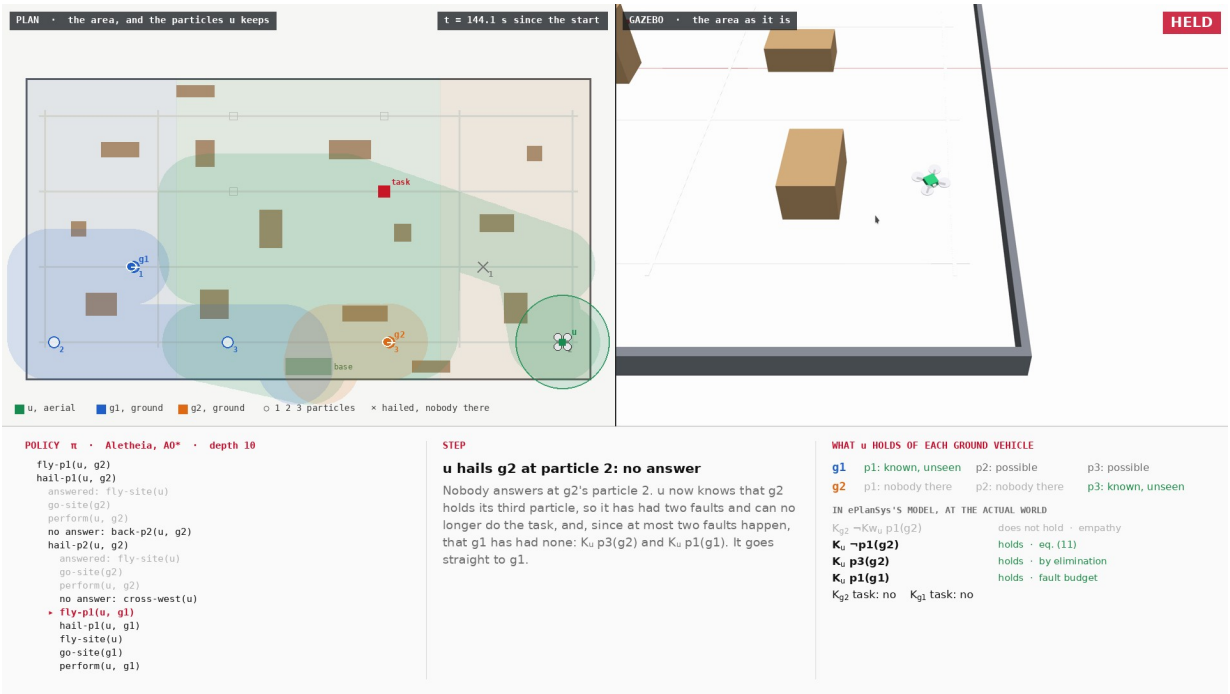


Figure 2: Seed 11 under Aletheia's policy, as the second hail at g2 finds nobody: the aerial vehicle now knows, without a visit, that g2 has had two faults and g1 none.

```
    go-site(g1)
    perform(u, g1)
      no answer: perform(u, g2)
    no answer: fly-site(u)
    perform(u, g2)
  no answer: back-p3(u, g2)
  hail-p3(u, g2)
  fly-site(u)
  go-site(g2)
  perform(u, g2)
```

Formulations. The policy minimises the number of steps of its worst branch and has no durations. The paper's procedure *as published* keeps each vehicle's particles on their own, with no joint budget and no assumption that some tool works, drops a vehicle whose attempt fails, and minimises the makespan of the next assignment at the most likely particle. The *strong* procedure keeps every pattern consistent with the hail answers, the attempts, the budget and the assumption that some tool works, which is the first-order knowledge u holds in the Kripke model, and chooses the assignment (one vehicle or both, as the paper's integer program allows) and the order of visits of least mean makespan over those patterns, attempting in order of arrival. *Ideal* is the strong procedure knowing where every vehicle is. The *timed* policy comes from a timed AND-OR search written for this report (`tools/timed_policy.py`) over the same grounded task: its states are the pointed models under the product update, contracted to their bisimulation quotient; an action applies where its designated events cover the designated worlds, so that knowledge preconditions gate it; its branches are the designated events weighted by the patterns that take them; and each node carries a clock with the durations of the kinematic model. It minimises expected time, the worst case breaking ties, and returns a policy in Aletheia's format that ePlanSys's plugin validates and its executor runs. The models it can reach and the transitions between them do not depend on the site or the clock; they are computed once per task (about 156 s for the variant, 20 s for the replica) and kept with the package as the task's skeleton. When the task is found, the plugin calls the search in place of Aletheia, with the site: 0.37 s from request to policy in the variant, against

15.0 s for Aletheia, both within the mission time. The policy and the strong procedure therefore differ in how they decide and not in what they know: a contingent policy least in worst-case steps against a greedy mean makespan estimated again after each observation. The gap between the published and the strong procedure is what the joint model contributes.

formulation	done	mean (s)	worst (s)	predicted (s)	hails	failed
ePlanSys, Aletheia	24/24	326.2	581.2	331.7 / 610.8	2.75	6
ePlanSys, timed	24/24	265.1	423.6	269.8 / 435.2	2.62	6
procedure, as published	24/24	273.3	436.3	280.9 / 446.4	1.83	6
procedure, strong	24/24	261.0	431.9	267.3 / 443.4	2.58	6
ideal	24/24	230.7	344.5	235.6 / 349.8	–	7

Table 7: The variant, per formulation, over the 24 seeds in the simulator; predicted: mean and worst in the kinematic model of `tools/estimate.py`.

Every formulation did the task in every run. The kinematic model predicted the order of the rows and their means within 8 s. No formulation can know a tool before it is tried: each attempted with a vehicle whose tool had failed in 6 or 7 runs. Seed by seed, Aletheia's policy was sooner than the strong procedure in 6 of 24 seeds and later in the rest; the timed policy was sooner in 4, within 1 s in 11, and 4.1 s later on average.

Sources of the gap. Replaying the policy with the kinematic model of `tools/estimate.py`, first with attempts in order of arrival and then fetching the other vehicle where the policy sends one with two drive faults, splits the gap to the strong procedure into three parts (Table 8). The first two come from the missing time model and the third from the objective, chiefly the recruiting of a second vehicle where the mean makespan would skip it. Aletheia has no action costs, and steps of fixed length would not repair this: a sequential policy adds up the legs of concurrent drives, so depth would follow total work and not makespan, and would penalise recruiting both vehicles. Aletheia has no temporal epistemic planning; the timed search supplies it for this task.

The timed search keeps the knowledge and changes the objective, and the parts of the gap close with it: in the kinematic model it is predicted at 269.8 s on average and 435.2 s at worst, against 267.3 and 443.4 s for the strong procedure. What keeps it from doing better is the domain's movement, not its reasoning: u reaches a second or third particle only along the route from the first, while the procedure, in the same model, flies to any particle directly. In a copy of the domain (`variants/belief-empathy-latent-direct.epddl`) where u may fly to a vehicle's second or third particle from anywhere, the site included, once it knows the particles before it empty, as the simulated aerial vehicle can, the timed policy is predicted at 267.8 s on average and 443.7 s at worst over all 48 pairs of site and pattern, against 269.0 and 454.4 s for the strong procedure. The copy was not run in the simulator.

part of the mean gap	seconds	from
attempts in a fixed order	23.5	the missing time model
a vehicle with two drive faults sent	22.6	the missing time model
the rest: recruiting where the mean would not	18.3	the objective
the gap, predicted	64.4	
the gap, measured in the simulator	65.2	

Table 8: The mean gap between Aletheia's policy and the strong procedure, taken apart in the kinematic model.

13 The least expected time

The timed search's value at the root is the least expected time to the task of any policy built from its moves (seeking a vehicle at a particle, sending a vehicle met, trying a vehicle sent) over what

seed	faults	site	Aletheia	timed	publ.	strong	ideal
1	g1 sound, g2 sound	(-4, 2)	194	193	193	193	169
2	g1 sound, g2 tool	(4, 2)	246	241	353	245	237
3	g1 sound, g2 1 drive	(4, 6)	340	268	268	268	261
4	g1 sound, g2 1 drive + tool	(-4, 6)	257	275	220	266	218
5	g1 sound, g2 2 drive	(-4, 2)	229	220	193	193	169
6	g1 tool, g2 sound	(4, 2)	254	201	195	195	193
7	g1 tool, g2 1 drive	(4, 6)	350	355	365	355	272
8	g1 1 drive, g2 sound	(-4, 6)	352	266	298	266	263
9	g1 1 drive, g2 tool	(-4, 2)	271	252	324	251	192
10	g1 1 drive, g2 1 drive	(4, 2)	311	316	319	284	208
11	g1 1 drive + tool, g2 sound	(4, 6)	434	239	218	218	216
12	g1 2 drive, g2 sound	(-4, 6)	581	265	298	266	263
13	g1 sound, g2 sound	(4, 6)	272	219	218	218	216
14	g1 sound, g2 tool	(-4, 6)	242	253	220	246	218
15	g1 sound, g2 1 drive	(-4, 2)	223	213	193	193	169
16	g1 sound, g2 1 drive + tool	(4, 2)	307	240	245	251	237
17	g1 sound, g2 2 drive	(4, 6)	575	268	268	268	261
18	g1 tool, g2 sound	(-4, 6)	319	266	344	266	263
19	g1 tool, g2 1 drive	(-4, 2)	388	386	398	392	265
20	g1 1 drive, g2 sound	(4, 2)	388	201	195	200	193
21	g1 1 drive, g2 tool	(4, 6)	426	424	436	432	344
22	g1 1 drive, g2 1 drive	(-4, 6)	352	355	358	355	304
23	g1 1 drive + tool, g2 sound	(-4, 2)	277	244	243	243	214
24	g1 2 drive, g2 sound	(4, 2)	241	201	195	200	193

Table 9: Mission time in seconds per seed in the simulator.

the aerial vehicle knows, every fault pattern weighted alike, with the durations of the kinematic model. In the replica, and in the variant's domain with direct movement, those moves include every decision the paper's procedure and the strong procedure make, so the value bounds what any procedure over the same knowledge reaches on average. `tools/timed_bound.py` computes it for every fault pattern at every site, times every formulation there by the same model, and checks that the bound holds at each site (Table 10).

formulation	replica	worst	over	variant	worst	over
least expected time (the timed search)	251.2	369.3	0.0	267.8	443.7	0.0
ePlanSys, timed, in the domain as written	251.2	369.3	0.0	271.3	454.4	+3.5
procedure, strong	–	–	–	269.0	454.4	+1.2
the paper's procedure, as published	254.3	369.3	+3.1	281.2	446.5	+13.4
ePlanSys, Aletheia	272.2	363.1	+21.0	327.8	610.8	+60.0
ideal: every robot heard	224.1	309.7	−27.1	235.0	382.4	−32.8

Table 10: Mean and worst time to the task in seconds in the kinematic model, every fault pattern at every site (24 cases in the replica, 48 in the variant), and the mean above the least expected time any policy over the aerial vehicle's knowledge reaches. In the variant the bound is the timed search with direct movement.

In the replica the timed policy is the bound itself. The paper's procedure is 3.1 s above it on average: it chooses one hail at a time, by the makespan it would have if the vehicle were at the particle hailed, where the search weighs every continuation over the fault patterns still possible. Aletheia's policy, the same knowledge without durations, is 21.0 s above it. In the variant the

strong procedure comes within 1.2 s of the bound, which is all that planning the whole contingent tree adds over its greedy estimate; the procedure as published is 13.4 s above, 12.2 s of it the joint inference it lacks; Aletheia is 60.0 s above. Of the 92.8 s between Aletheia and the ideal method, 56.5 s is time, which the timed search recovers over the same model; 3.5 s is the domain's movement, which the direct-movement copy recovers; and 32.8 s, the distance from the bound to the ideal method, is what the aerial vehicle does not know. That last part is the price of being out of contact: no policy over the same knowledge recovers any of it, and only an observation or a message could. The bound is on the mean. Asked for the worst case first, the same search returns a policy that finishes every pattern at every site within 432.8 s, against 454.4 s for the strong procedure, at 12.7 s more on average. A bound on the worst case is a property of the whole policy, every branch at once, which a procedure choosing one assignment at a time does not compute.

14 Larger teams

The same domain with n ground vehicles, the first half covering the west, the site in the east and the same fault budget has $1 + 2n + n(n-1)/2$ fault patterns (Table 11).

robots	ground	patterns	actions	depth	expanded	time (s)
3	2	6	54	10	5848	0.7
4	3	10	100	9	4609	1.0
5	4	15	160	9	5588	2.8
6	5	21	234	9	5917	6.4
7	6	28	322	9	6108	12.6
8	7	36	424	9	6127	23.6
9	8	45	540	9	6137	53.4

Table 11: Aletheia's AO* with consistent beliefs on two threads, for teams of one aerial vehicle and n ground vehicles.

Every team was solved, from 0.7 s for 2 ground vehicles to 53.4 s for 8, the paper's largest team with the aerial vehicle. The time roughly doubles with each vehicle added, while the expansions stay near 6 000: the models grow with the fault patterns while the policy barely does. Its depth falls from 10 to 9 once a second vehicle covers the aerial vehicle's half, where a neighbour can be sought without crossing. The paper's allocation is polynomial; this search is exhaustive, and the budget of two faults is what keeps its fault patterns quadratic in the team.

15 Implementation defects

ePlanSys's executor dispatches together consecutive actions only when the agents they name are disjoint. The drive to the site named u as well as the vehicle, and it ran after u's flight to the site instead of with it; it now names the vehicle alone, and the policy is unchanged. The demonstration's function that finds a point along a path tested for the last segment by identity, and Python shares equal tuple constants, so a path that passed a point twice ended at its first visit; it stalled the flock at a turn. The first batch found a third: detection within 1.2 m of a site was narrower than the 4 m between lanes assumes, so that in the flock a site on a street was seen only by the ground vehicle driving it, and when that vehicle left with two faults the task was never found. Detection is now 2.2 m, the half-width of the lanes, and the batch was run again in full. The fourth was in the timed search's clock: in the replica a ground vehicle answers a drive to the site only on arriving, and the executor, which waits for each group of consecutive actions on disjoint agents, held the aerial vehicle until then, so that where the search had sent one vehicle and flown on to recruit the other, u waited at the first vehicle's particle (seed 6: 323.6 s against 250.5 s planned). The search and the kinematic model now carry that hold; the twelve runs made before the change are kept apart and not reported.

16 Limitations

The recreation is of the paper's scenario and not of its whole framework: one task, one aerial vehicle, a partition computed once, and known coverage routes. Faults happen before the task can be found. A vehicle with two faults is taken to have lost the capability, where the paper only slows it, so that the policy has a reason to prefer one vehicle without durations. Aletheia's AO* minimises the depth of the worst branch, not time: where the paper's procedure, which estimates makespan, chooses otherwise, the difference is the metric and not what either knows. The executor is one process; the task is found by proximity in the simulator's poses; mission time includes the planner's wall time, and the batch ran six simulations at once. The timed search was written for this report and is not part of Aletheia: its moves are this task's (seek, send, try), each a fixed sequence of the domain's actions, and its durations come from the kinematic model the strong procedure estimates with, so that the two share their idea of time and the simulator tests both. It is not a domain-independent temporal planner. The bound of Section 13 holds in the kinematic model over the moves the search considers, and the direct-movement domain was compared in that model only.

References

[1] L. Bramblett and N. Bezzo. Epistemic planning for heterogeneous robotic systems. arXiv:2308.00579, 2023.

[2] L. Bramblett, S. Gao and N. Bezzo. Epistemic prediction and planning with implicit coordination for multi-robot teams in communication restricted environments. *Proceedings of ICRA*, 2023.

[3] H. van Ditmarsch, W. van der Hoek and B. Kooi. *Dynamic Epistemic Logic*. Springer, 2007.

[4] A. Baltag, L. S. Moss and S. Solecki. The logic of public announcements, common knowledge, and private suspicions. *Proceedings of TARK VII*, 43–56, 1998.

[5] T. Bolander and M. B. Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics* 21(1):9–34, 2011.

[6] T. Engesser, T. Bolander, R. Mattmüller and B. Nebel. Cooperative epistemic multi-agent planning for implicit coordination. *Proceedings of the Ninth Workshop on Methods for Modalities*, EPTCS 243, 75–90, 2017.

[7] N. J. Nilsson. *Principles of Artificial Intelligence*. Tioga, 1980.